

React.js Project Setup

npm • Vite • Files • Modules • Components

Goal: Understand what each important file and command does — not just memorize commands.

Learning Roadmap

- 1. What are Node.js and npm?
- 2. Why does a React project need npm?
- 3. What is Vite and why use it?
- 4. React installation — step by step
- 5. Folder structure and important files
- 6. package.json, ESLint and vite.config
- 7. Modules, import and export
- 8. Components and how React uses them
- 9. Vite alternatives and when to use them

What is Node.js?

- Node.js lets JavaScript run outside the browser.
- React development tools run on your computer using Node.js.
- Node.js provides the environment needed to run commands such as npm.
- Think: Browser → JavaScript in the browser; Node.js → JavaScript on your computer.

Easy example: Node.js is like the engine that lets JavaScript tools run on your computer.

What is npm?

- npm means Node Package Manager.
- It comes with Node.js when Node.js is installed.
- npm helps us install, update and manage packages (libraries/tools).
- Example packages: React, Vite, ESLint and many others.
- npm also runs project commands written in package.json.

Easy example: npm is like an app store + manager for JavaScript project packages.

Why do we need npm in React?

- A React project normally uses many packages.
- npm downloads those packages into the project.
- npm keeps track of package versions.
- npm can run commands such as development server and production build.
- Without a package manager, installing and maintaining project dependencies would be difficult.

```
npm install  
npm run dev  
npm run build
```

Where is npm used?

- Terminal / Command Prompt: we type npm commands here.
- package.json: stores project scripts and package information.
- node_modules: contains installed packages.
- package-lock.json: records the exact dependency tree installed by npm.
- npm is mainly a development/project tool; it is not a React component.

What is a Package?

- A package is reusable code made available to other projects.
- React itself is installed as an npm package.
- Vite is installed/used through npm tooling.
- Instead of writing everything from zero, we reuse packages.
- Packages can have their own dependencies.

Simple idea: Package = ready-made code/tool that our project can use.

What is Vite?

- Vite is a modern frontend build tool.
- It gives us a fast development server.
- It helps process modern frontend code.
- It creates an optimized production build.
- It is not React itself — React and Vite have different jobs.

Remember: React helps us build UI. Vite helps us develop and build the project.

React + Vite: Who Does What?

- React → components, JSX, state, props and UI.
- Vite → development server and project build process.
- npm → installs packages and runs project scripts.
- Browser → finally displays the application.

You write React code



npm manages packages



Vite runs/builds the project



Browser displays the result

Why use Vite for React?

- Very fast development startup.
- Fast updates while coding (Hot Module Replacement / HMR).
- Simple project setup.
- Good support for JavaScript and TypeScript.
- Modern React projects can be started quickly.

Student tip: You can learn React without first becoming an expert in Vite.

React Installation — Step 1: Check Node.js

- Install a current supported Node.js version from the official Node.js website.
- Open Terminal / Command Prompt.
- Check Node.js version.
- Check npm version.

```
node -v  
npm -v
```

Expected: Both commands should print a version number.

React Installation — Step 2: Create Project

- Open Terminal in the folder where you want the project.
- Run the Vite project creation command.
- Choose React when asked.
- Choose JavaScript for a beginner React course.

```
npm create vite@latest my-react-app
```

React Installation — Step 3: Choose Options

- Project name → my-react-app (or your own name).
- Framework → React.
- Variant → JavaScript.
- TypeScript is another option, but JavaScript is easier for first-time React learners.

```
Framework: React  
Variant: JavaScript
```

React Installation — Step 4: Enter Project

- Move into the newly created project folder.
- Install all dependencies listed in package.json.
- Start the development server.

```
cd my-react-app  
npm install  
npm run dev
```

React Installation — Step 5: Open in Browser

- `npm run dev` starts the local development server.
- The terminal shows a local address.
- Open that address in your browser.
- Edit a React file and save it — the browser updates quickly.

```
npm run dev
```

```
Local: http://localhost:5173/
```

Important Commands

- `npm create vite@latest` → create a new Vite project.
- `npm install` → install project dependencies.
- `npm run dev` → start development server.
- `npm run build` → create production build.
- `npm run preview` → preview the production build locally.
- `npm install package-name` → add another package.

Basic React Project Folder Structure

- my-react-app/ → main project folder
- node_modules/ → installed packages (usually do not edit)
- public/ → files served as public/static assets
- src/ → your main React source code
- index.html → HTML entry page
- package.json → project information, scripts and dependencies
- package-lock.json → exact installed dependency versions/tree
- vite.config.js → Vite configuration
- eslint.config.js → ESLint configuration

src Folder — Where We Write React

- `src/main.jsx` → starts the React application and renders the root component.
- `src/App.jsx` → commonly the main/root UI component in a simple project.
- `src/index.css` → global CSS (name can vary by template).
- Create folders such as `components/`, `pages/`, `assets/` as the project grows.

Important: Most of your daily React coding happens inside `src/`.

What is package.json?

- package.json is the project's information and configuration file.
- It contains project name/version and other metadata.
- It contains dependencies your project needs.
- It contains scripts that npm can run.
- npm reads this file to understand how the project is managed.

```
{  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build"  
  },  
  "dependencies": {  
    "react": "..."  
  }  
}
```

package.json — dependencies vs devDependencies

- dependencies → packages needed by the application.
- devDependencies → tools mainly needed while developing/building.
- Example: React is normally a dependency.
- Example: ESLint and Vite tooling may be listed as development dependencies.
- Do not manually change versions unless you understand the effect.

What is package-lock.json?

- npm creates package-lock.json when installing packages.
- It records the exact dependency versions/tree used for the project.
- It helps teammates and CI install a consistent dependency tree.
- Usually commit it to Git for an application project.
- Do not edit it manually in normal development.

Easy idea: package.json says what we need; package-lock.json records what npm actually locked.

What is node_modules?

- node_modules contains installed npm packages.
- npm install creates/updates it.
- It can become very large.
- Usually it is not committed to Git.
- If deleted, npm install can recreate it from package.json/package-lock.json.

```
npm install
  ↓
node_modules/
  ↓
React + Vite + other packages
```

What is ESLint?

- ESLint is a code quality/linting tool.
- It checks JavaScript/JSX code for common mistakes and style problems.
- It can warn you before a small coding mistake becomes a bigger problem.
- It is not React and it does not run your application.

Easy example: ESLint is like a teacher checking your code and pointing out possible mistakes.

ESLint Configuration

- Modern Vite React templates commonly include an ESLint configuration file.
- The filename may be `eslint.config.js` in current templates.
- It tells ESLint which rules/plugins/configuration to use.
- Rules can be changed as a project needs.

```
eslint.config.js
```

```
ESLint
```

```
↓
```

```
checks source code
```

```
↓
```

```
warnings / errors
```

What is vite.config.js?

- vite.config.js is Vite's configuration file.
- It lets us customize how Vite behaves.
- Examples: plugins, aliases, development server settings and build options.
- Small projects may need very little configuration.
- Do not change settings randomly; first understand why the setting is needed.

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
})
```

What is index.html?

- It is the HTML page used as the entry point for the web app.
- It contains the root element where React will be mounted.
- The script entry points to your React application, such as `/src/main.jsx`.
- React then controls the UI inside the root element.

```
<body>  
  <div id="root"></div>  
  <script type="module" src="/src/main.jsx"></script>  
</body>
```

What is main.jsx?

- main.jsx is commonly the starting point of a Vite React application.
- It imports React-related code and the App component.
- It finds the HTML root element.
- It renders the React component tree into that root.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode><App /></StrictMode>
)
```

What is a Module?

- A module is a file that contains code which can be shared with other files.
- In modern JavaScript, files can export values and import values.
- This helps divide a large project into small, understandable files.
- React projects use modules heavily.

Easy example: One file can provide a function/component; another file can import and use it.

Why use Modules?

- Keep code organized.
- Reuse code in different files.
- Avoid putting the whole application into one huge file.
- Make components and utilities easier to maintain.
- Control what a file makes available using export.

```
Header.jsx → exports Header
              ↓
App.jsx    → imports Header
```

import and export — Named Export

- A named export uses a name.
- You can export multiple named values from one file.
- When importing, use curly braces { }.
- The imported name normally must match the exported name.

```
// math.js
export const add = (a, b) => a + b
export const sub = (a, b) => a - b

// App.jsx
import { add, sub } from './math.js'
```

import and export — Default Export

- A file can have one default export.
- When importing a default export, curly braces are not used.
- You can choose the local import name.

```
// Header.jsx
function Header() {
  return <h1>Header</h1>
}
export default Header

// App.jsx
import Header from './Header.jsx'
```

Default + Named Export Together

- A file can have one default export plus named exports.
- Use braces only for named exports.
- This is common when a file has one main thing plus helper exports.

```
// tools.js
export default function mainTool() {}
export const version = '1.0'

import mainTool, { version } from './tools.js'
```

Common Import Forms

- Default import → `import App from './App.jsx'`
- Named import → `import { useState } from 'react'`
- Multiple named imports → `import { add, sub } from './math.js'`
- Namespace import → `import * as Utils from './utils.js'`
- Side-effect import → `import './index.css'`

Rule to remember: Default import → no braces. Named import → braces.

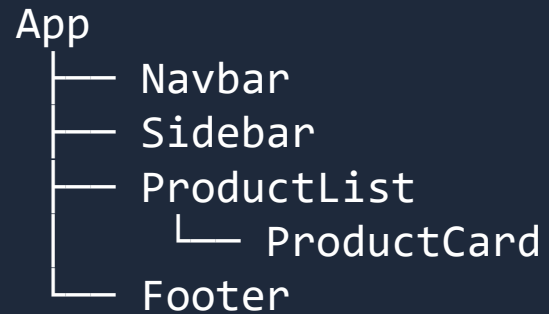
What is a Component?

- A component is a reusable piece of UI.
- It is usually a JavaScript function that returns JSX.
- Examples: Header, Navbar, Button, ProductCard, LoginForm.
- Components help us split a large UI into small parts.

```
function Welcome() {  
  return <h1>Welcome to React</h1>  
}  
  
export default Welcome
```

Why Components?

- Reuse → create once, use many times.
- Organization → each UI part has a clear job.
- Maintenance → change one component instead of many repeated blocks.
- Teamwork → different developers can work on different components.



Component Usage

- Import the component file.
- Use the component like an HTML tag.
- Component names normally start with a capital letter.
- Pass data using props.

```
import Header from './components/Header.jsx'

function App() {
  return (
    <>
      <Header />
      <h1>Home Page</h1>
    </>
  )
}
```

Vite Alternatives

- Parcel → simple, low-configuration bundler/build tool.
- Webpack → very configurable and widely used in older/existing projects.
- Rspack → high-performance Rust-based bundler with strong Webpack compatibility.
- Rsbuild → higher-level build tool built around Rspack.
- Turbopack → high-performance bundler associated strongly with the Vercel/Next.js ecosystem.
- Next.js → React framework, not simply a build tool.

Vite vs Alternatives — Easy Comparison

- Vite → excellent default for learning modern React.
- Parcel → great when you want very little configuration.
- Webpack → important to understand for legacy/enterprise projects.
- Rspack → useful for performance + Webpack ecosystem compatibility.
- Rsbuild → modern, fast, higher-level experience around Rspack.
- Turbopack → modern high-performance bundling, especially in Next.js ecosystem.
- Next.js → choose when you need a full React framework.

Which One Should Learn First?

- Start with React + Vite.
- Do not learn five build tools at the same time.
- First understand React concepts: components, JSX, props, state, events and hooks.
- Then understand what a build tool does.
- Later, learn alternatives when you encounter them in real projects.

Recommended learning path: React → Vite → React Router → API → real project → then explore other tooling.

Common Beginner Mistakes

- Thinking Vite is React — it is not.
- Thinking npm is a programming language — it is a package manager/tool.
- Editing `node_modules` manually.
- Deleting `package-lock.json` without knowing why.
- Putting every component inside `App.jsx`.
- Confusing default import with named import.
- Forgetting that component names normally start with a capital letter.
- Changing configuration files without understanding the reason.

Quick Revision — One Line Each

- Node.js → runs JavaScript tools outside the browser.
- npm → installs/manages packages and runs scripts.
- Vite → fast development/build tool.
- package.json → project information, scripts and dependencies.
- package-lock.json → locked dependency tree.
- node_modules → installed packages.
- ESLint → checks code for possible problems.
- vite.config.js → Vite customization.
- Module → a file that can export/import code.
- Component → reusable piece of React UI.

Final Mental Model

- Node.js → environment
- npm → package manager + script runner
- Vite → development server + build tool
- React → UI library
- Component → reusable UI part
- Module → file that shares code using import/export
- package.json → project instructions/dependencies
- ESLint → code checker
- vite.config.js → Vite settings

Remember: Learn the job of each tool/file. Do not memorize names without understanding what problem they solve.